

GoLisp – Tutorial

Dieses Dokument beschreibt die öffentlichen Funktionen, Spezialformen und Makros von GoLisp mit kurzen, lauffähigen Beispielen. Dateioperationen nutzen immer das Projekt-temp-Verzeichnis `./tmp`. Funktionen, die externe Dienste benötigen (sigo, PostgreSQL), enthalten beschreibende Beispiele.

Spezialformen

quote

Syntax: `(quote ausdruck)` bzw. `'ausdruck`

Verhindert die Auswertung eines Ausdrucks und gibt ihn als Datenstruktur zurück.

```
(quote (+ 1 2))  
'(+ 1 2)  
(car '(a b c))
```

if

Syntax: `(if bedingung dann [sonst])`

Wertet dann aus, wenn die Bedingung wahr ist, sonst den optionalen else-Zweig.

```
(if t 1 2)  
(if nil 'ja 'nein)  
(if (> 3 2) 'größer)
```

define

Syntax: `(define symbol wert)` bzw. `(setq symbol wert)`

Bindet einen Wert an einen Namen im aktuellen Environment.

```
(define x 10)
(setq y 20)
(+ x y)
```

setq

Syntax: (setq symbol wert)

Alias für define – bindet einen Wert an einen Namen.

```
(setq antwort 42)
```

defun

Syntax: (defun name (parameter ...) body ...)

Definiert eine benannte Funktion. Mehrere Body-Ausdrücke werden implizit in begin gewrappt.

```
(defun quadrat (x) (* x x))
(quadrat 7)
(defun add (a b) (print a) (+ a b))
```

lambda

Syntax: (lambda (parameter ...) body ...)

Erzeugt eine anonyme Funktion bzw. Closure.

```
((lambda (x) (* x 2)) 5)
(define doppel (lambda (x) (* 2 x)))
(doppel 4)
```

let

Syntax: (let ((var wert) ...) body ...)

Bindet Variablen parallel – alle Werte werden im äußeren Environment ausgewertet.

```
(let ((a 1) (b 2)) (+ a b))
(let ((x 5) (y 10)) (list x y))
```

let*

Syntax: (let* ((var wert) ...) body ...)

Bindet Variablen sequentiell – jede Bindung sieht die vorherigen.

```
(let* ((x 1) (y (+ x 1))) y)
(let* ((a 2) (b (* a 3))) (+ a b))
```

begin

Syntax: (begin ausdruck ...)

Wertet Ausdrücke nacheinander aus und gibt das Ergebnis des letzten zurück.

```
(begin (print 1) (print 2) 3)
(begin (define x 1) (set! x 2) x)
```

set!

Syntax: (set! symbol wert)

Ändert den Wert einer bestehenden Variablen.

```
(define a 1)
(set! a 99)
a
```

setq*

Syntax: (setq* var1 wert1 var2 wert2 ...)

Setzt mehrere Variablen sequentiell; fehlende Variablen werden neu angelegt.

```
(setq* a 1 b (+ a 1) c (+ b 1))
(list a b c)
```

defmacro

Syntax: (defmacro name (parameter ...) body ...)

Definiert ein Makro, das zur Expandierungszeit transformiert wird.

```
(defmacro unless (test . body) `(if ,test ()  
    (begin ,@body)))  
(unless nil 'ok)
```

macroexpand

Syntax: (macroexpand form)

Expandiert Makros einmal auf Top-Level und gibt das Ergebnis zurück.

```
(defmacro doppelt (x) `(+ ,x ,x))  
(macroexpand (doppelt 5))
```

macroexpand-all

Syntax: (macroexpand-all form)

Expandiert Makros rekursiv in allen Subformen.

```
(macroexpand-all (doppelt (+ 1 2)))
```

function

Syntax: (function ausdruck)

Wertet den übergebenen Ausdruck aus und gibt seinen Wert zurück.

```
(function +)
```

flet

Syntax: (flet ((name (parameter ...) body ...) ...) body ...)

Definiert lokale Funktionen, die über das äußere Environment schließen.

```
(flet ((inc (x) (+ x 1))) (inc 5))
```

labels

Syntax: (labels ((name (parameter ...) body ...) ...) body ...)

Wie flet, aber die lokalen Funktionen können sich gegenseitig rekursiv aufrufen.

```
(labels ((fac (n) (if (= n 0) 1 (* n (fac (- n 1))))))  
  (fac 5))
```

block

Syntax: (block name body ...)

Benannter Block; return-from kann ihn vorzeitig verlassen.

```
(block aus (return-from aus 42) (print 'nie))
```

return-from

Syntax: (return-from name [wert])

Verlässt den benannten Block nicht-lokal und liefert den optionalen Wert.

```
(block outer (return-from outer 99) 'unten)
```

mapcar

Syntax: (mapcar funktion liste)

Wendet eine Funktion auf jedes Element einer Liste an und liefert die Ergebnisliste.

```
(mapcar (lambda (x) (* x x)) '(1 2 3))  
(mapcar atom? '(1 (2 3) "a"))
```

load

Syntax: (load "datei.lisp")

Lädt eine Lisp-Datei, wertet alle enthaltenen Ausdrücke aus und gibt das letzte Ergebnis zurück.

```
(load "stdlib.lisp")
```

and

Syntax: (and ausdruck ...)

Wertet Ausdrücke von links nach rechts aus und bricht bei nil ab. Liefert den letzten wahren Wert.

```
(and 1 2 3)
(and t nil 'x)
(and)
```

or

Syntax: (or ausdruck ...)

Wertet Ausdrücke von links nach rechts aus und liefert den ersten wahren Wert, sonst nil.

```
(or nil 'a 'b)
(or nil nil)
(or 0 'x)
```

not

Syntax: (not x)

Gibt t zurück, wenn x falsch (nil) ist, sonst nil.

```
(not nil)
(not t)
(not 0)
```

cond

Syntax: (cond (test body ...) ... [(else body ...)])

Verzweigungskette; der erste wahre Test bestimmt den ausgeführten Body.

```
(cond ((= 1 2) 'nein) (t 'ja))
(cond ((> 5 3) 'größer) (else 'kleiner))
```

case

Syntax: (case schlüssel ((wert ...) body ...) ...
[(else body ...)])

Wählt einen Zweig durch strukturelle Gleichheit (equal?) des Schlüssels aus.

```
(case 'b ((a) 1) ((b c) 2) (else 3))  
(case 5 ((1 2 3) "klein") ((4 5 6) "mittel"))
```

while

Syntax: (while test body ...)

Wertet den Body so lange aus, wie der Test wahr ist. Gibt nil zurück.

```
(define i 0)  
(while (< i 3) (print i) (set! i (+ i 1))))
```

do

Syntax: (do ((var init schritt) ...) (test ergebnis) body ...)

Scheme-artige Schleife mit parallelen Bindungen, Abbruchtest und optionalem Body.

```
(do ((i 0 (+ i 1))) ((= i 3) i) (print i))  
(do ((x 1 (* x 2)) (n 0 (+ n 1))) ((> x 10) n))
```

quasiquote

Syntax: (quasiquote ausdruck) bzw. `ausdruck``

Wie quote, erlaubt aber unquote (,x) und unquote-splice (,@x) innerhalb.

```
(define x 5)  
(`(liste ,x)  
(a ,@'(b c)))
```

eval

Syntax: (eval ausdruck)

Wertet einen bereits ausgewerteten Ausdruck nochmals im globalen Environment aus.

```
(eval '(+ 1 2))  
(define code '(+ 3 4))  
(eval code)
```

catch

Syntax: (catch body handler)

Fängt Fehler im Body ab und ruft handler mit der Fehler-Cell auf.

```
(catch (error "ups") (lambda (e) (list 'fehler e)))  
(catch (/ 1 0) (lambda (e) 'recovered))
```

parfunc

Syntax: (parfunc ergebnis [:timeout n] ausdruck ...)

Wertet alle Ausdrücke parallel aus, speichert die Ergebnisliste in ergebnis und gibt sie zurück.

```
(parfunc ergebnis (+ 1 2) (* 3 4) (- 10 5))  
(car ergebnis)
```

lock

Syntax: (lock mutex ausdruck ...)

Führt die Ausdrücke atomar unter dem angegebenen Mutex aus.

```
(define m (lock-make))  
(define x 0)  
(lock m (set! x (+ x 1)) (set! x (+ x 1)))  
x
```

Arithmetik

+

Syntax: (+ zahl ...)

Addiert alle übergebenen Zahlen.

```
(+ 1 2 3)  
(+ 5.5 4.5)
```


-

Syntax: (- zahl ...) bzw. (- x y)

Subtrahiert die folgenden Zahlen von der ersten. Ohne Argumente liefert es 0.

```
(- 10 3)
```

```
(- 5)
```

*

Syntax: (* zahl ...)

Multipliziert alle übergebenen Zahlen.

```
(* 2 3 4)
```

```
(* 7)
```

/

Syntax: (/ x y)

Dividiert x durch y. Fehler bei Division durch 0.

```
(/ 10 2)
```

```
(/ 1 4)
```

mod

Syntax: (mod x y)

Liefert den Rest der Division x durch y (Gleitkomma).

```
(mod 10 3)
```

```
(mod 7.5 2.5)
```

remainder

Syntax: (remainder x y)

Alias für mod.

```
(remainder 10 3)
```

abs

Syntax: (abs x)

Liefert den absoluten Betrag einer Zahl.

```
(abs -5)  
(abs -3.14)
```

random

Syntax: (random) bzw. (random n)

Liefert eine Zufallszahl. Ohne Argument eine große ganze Zahl, mit n einen Wert von 0 bis n-1.

```
(random 6)  
(random 100)
```

Vergleiche

=

Syntax: (= x y)

Numerische Gleichheit.

```
(= 5 5)  
(= 1 2)
```

<

Syntax: (< x y)

Numerisch kleiner.

```
(< 2 5)  
(< 5 2)
```

>

Syntax: (> x y)

Numerisch größer.

```
(> 5 2)
```

```
(> 2 5)
```

>=

Syntax: (**>=** x y)

Numerisch größer oder gleich.

```
(>= 5 5)
```

<=

Syntax: (**<=** x y)

Numerisch kleiner oder gleich.

```
(<= 3 7)
```

equal?

Syntax: (equal? x y)

Strukturelle Gleichheit (rekursiv für Listen).

```
(equal? '(1 2) '(1 2))
```

```
(equal? 'a 'a)
```

```
(equal? 5 5)
```

eq

Syntax: (eq x y)

Pointer-Identität – prüft, ob beide Argumente dasselbe Objekt im Speicher sind.

```
(eq nil nil)
```

```
(eq (list) (list))
```

```
(eq 'foo 'foo)
```

eq?

Syntax: (eq? x y)

Alias für eq.

```
(eq? nil nil)
```

Typ-Prädikate

string?

Syntax: (string? x)

Gibt t zurück, wenn x ein String ist.

```
(string? "hallo")  
(string? 42)
```

number?

Syntax: (number? x)

Gibt t zurück, wenn x eine Zahl ist.

```
(number? 42)  
(number? "42")
```

list?

Syntax: (list? x)

Gibt t zurück, wenn x eine Liste oder nil ist.

```
(list? '(1 2))  
(list? nil)  
(list? 5)
```

symbol?

Syntax: (symbol? x)

Gibt t zurück, wenn x ein Symbol (Atom) ist.

```
(symbol? 'foo)  
(symbol? "foo")
```

atom?

Syntax: (atom? x)

Alias für atom – gibt t zurück, wenn x kein Cons/Liste ist.

```
(atom? 5)
(atom? '(1 2))
```

null?

Syntax: (null? x)

Alias für null – gibt t zurück, wenn x nil ist.

```
(null? nil)
(null? '(1))
```

atom

Syntax: (atom x)

Gibt t zurück, wenn x kein Cons/Liste ist.

```
(atom 5)
(atom 'x)
(atom '(1 2))
```

null

Syntax: (null x)

Gibt t zurück, wenn x nil ist.

```
(null nil)
(null '())
(null '(1))
```

Listen

car

Syntax: (car liste)

Liefert das erste Element einer Liste.

```
(car '(a b c))
(car '(1 2))
```

cdr

Syntax: (cdr liste)

Liefert die Restliste ohne das erste Element.

```
(cdr '(a b c))  
(cdr '(1))
```

cons

Syntax: (cons x liste)

Fügt x als neues Kopfelement vor eine Liste.

```
(cons 1 '(2 3))  
(cons 'a nil)
```

list

Syntax: (list element ...)

Erzeugt eine neue Liste aus den Argumenten.

```
(list 1 2 3)  
(list 'a 'b)
```

append

Syntax: (append liste element)

Hängt ein einzelnes Element an das Ende einer Liste an.

```
(append '(1 2) 3)  
(append '(a) 'b)
```

apply

Syntax: (apply funktion arg1 ... liste)

Wendet eine Funktion an; das letzte Argument wird als Liste gesplitt.

```
(apply + '(1 2 3))  
(apply + 1 2 '(3 4))
```

funcall

Syntax: (funcall funktion arg ...)

Ruft eine Funktion mit den angegebenen Argumenten auf.

```
(funcall + 1 2 3)
(funcall (lambda (x) (* x x)) 6)
```

Strings

string-length

Syntax: (string-length str)

Liefert die Anzahl der Zeichen (Runes) eines Strings.

```
(string-length "Hallo")
(string-length "αβγ")
```

string-append

Syntax: (string-append str ...)

Verkettet mehrere Strings zu einem neuen String.

```
(string-append "Hallo" " " "Welt")
(string-append "a" "b" "c")
```

substring

Syntax: (substring str start end)

Liefert den Teilstring von start (inklusive) bis end (exklusiv).

```
(substring "Hallo" 0 2)
(substring "Welt" 1 3)
```

string-upcase

Syntax: (string-upcase str)

Wandelt einen String in Großbuchstaben um.

```
(string-upcase "hallo")
```

string-downcase

Syntax: (string-downcase str)

Wandelt einen String in Kleinbuchstaben um.

```
(string-downcase "HALLO")
```

string->number

Syntax: (string->number str)

Parst einen String als Zahl.

```
(string->number "42")  
(string->number "3.14")
```

number->string

Syntax: (number->string zahl)

Wandelt eine Zahl in einen String um.

```
(number->string 42)  
(number->string 3.14)
```

string->list

Syntax: (string->list str)

Wandelt einen String in eine Liste von Einzelzeichen-Strings um.

```
(string->list "abc")
```

list->string

Syntax: (list->string liste)

Verkettet eine Liste von Strings zu einem String.

```
(list->string '("a" "b" "c"))
```

string-replace

Syntax: (string-replace str alt neu)

Ersetzt alle Vorkommen von alt durch neu.

```
(string-replace "Hallo Welt" "Welt" "GoLisp")
```

string-trim

Syntax: (string-trim str)

Entfernt führende und abschließende Leerzeichen.

```
(string-trim "  hallo  ")
```

string-contains

Syntax: (string-contains str sub)

Gibt t zurück, wenn sub in str enthalten ist.

```
(string-contains "Hallo" "all")  
(string-contains "Hallo" "xyz")
```

Ein-/Ausgabe

print

Syntax: (print wert ...)

Gibt Werte ohne abschließenden Zeilenumbruch auf stdout aus. Rückgabewert ist das letzte Argument.

```
(print "Hallo")  
(print 1 " " 2)
```

println

Syntax: (println wert ...)

Gibt Werte mit abschließendem Zeilenumbruch auf stdout aus. Rückgabewert ist das letzte Argument.

```
(println "Zeile")  
(println 1 2 3)
```

read

Syntax: (read str)

Parst einen String und gibt die entsprechende Lisp-Datenstruktur zurück.

```
(read "(+ 1 2)")  
(car (read "(a b c)"))
```

Fehler

error

Syntax: (error wert)

Signalisiert einen Lisp-Laufzeitfehler, der von catch aufgefangen werden kann.

```
(error "etwas ist schief")  
(catch (error 'x) (lambda (e) (println e)))
```

Makro-Hilfe

gensym

Syntax: (gensym)

Erzeugt ein eindeutiges Symbol für Makros.

```
(gensym)  
(gensym)
```

Datei-I/O

file-write

Syntax: (file-write "pfad" inhalt ...)

Schreibt Inhalte in eine Datei (überschreibt).

```
(system "mkdir -p ./tmp")  
(file-write "./tmp/demo.txt" "Hallo")  
(file-write "./tmp/zahlen.txt" 1 " " 2)
```

file-append

Syntax: (file-append "pfad" inhalt ...)

Hängt Inhalte an eine Datei an.

```
(system "mkdir -p ./tmp")  
(file-write "./tmp/demo.txt" "Hallo")  
(file-append "./tmp/demo.txt" " Welt")
```

file-read

Syntax: (file-read "pfad")

Liest den Inhalt einer Datei als String.

```
(system "mkdir -p ./tmp")  
(file-write "./tmp/demo.txt" "Hallo")  
(file-read "./tmp/demo.txt")
```

file-exists?

Syntax: (file-exists? "pfad")

Gibt t zurück, wenn die Datei existiert.

```
(system "mkdir -p ./tmp")  
(file-exists? "./tmp/demo.txt")  
(file-exists? "./tmp/nichtda.txt")
```

file-delete

Syntax: (file-delete "pfad")

Löscht eine Datei. Gibt t zurück.

```
(system "mkdir -p ./tmp")  
(file-write "./tmp/demo.txt" "x")  
(file-delete "./tmp/demo.txt")
```

Nebenläufigkeit

chan-make

Syntax: (chan-make) bzw. (chan-make n)

Erzeugt einen Channel (optional mit Puffergröße n).

```
(define ch (chan-make))  
(define ch2 (chan-make 10))
```

chan-send

Syntax: (chan-send ch wert)

Sendet einen Wert in einen Channel.

```
(chan-send ch 42)
```

chan-recv

Syntax: (chan-recv ch)

Empfängt einen Wert aus einem Channel (blockierend, wenn leer).

```
(chan-recv ch)
```

lock-make

Syntax: (lock-make)

Erzeugt einen Mutex für lock.

```
(define mu (lock-make))
```

KI

sigo

Syntax: (sigo prompt [modell] [session-id] [host])

Sendet einen Prompt an sigoREST und gibt die Antwort als String zurück. Default-Modell und Host können via Env-Variablen gesetzt werden.

```
(sigo "Erkläre TCO")  
(sigo "Hallo" "cl46-s")
```

sigo-models

Syntax: (sigo-models)

Liefert eine Liste der verfügbaren sigoREST-Modelle.

```
(sigo-models)
```

sigo-host

Syntax: (sigo-host "http://...")

Setzt oder liest den sigoREST-Host.

```
(sigo-host "http://mammoth:9080")
```

Zeit

sleep

Syntax: (sleep ms)

Pausiert für die angegebene Anzahl Millisekunden.

```
(sleep 100)  
(sleep 2000)
```

Memory

memstats

Syntax: (memstats)

Liefert Go-Runtime-Memory-Statistiken als Assoziationsliste.

```
(memstats)
(assoc 'heapalloc (memstats))
```

Shell & System

system

Syntax: (system "shell-kommando")

Führt ein Shell-Kommando aus und gibt den Exit-Code zurück (0 = OK).

```
(system "echo hallo")
(system "test -f ./tmp/demo.txt")
```

file-stat

Syntax: (file-stat "pfad")

Liefert eine Assoziationsliste mit size und mtime oder nil, wenn die Datei nicht existiert.

```
(system "mkdir -p ./tmp")
(file-write "./tmp/demo.txt" "Hallo")
(file-stat "./tmp/demo.txt")
(assoc 'size (file-stat "./tmp/demo.txt"))
```

assoc

Syntax: (assoc key alist)

Sucht das erste Paar (key . val) in einer Assoziationsliste mittels equal?.

```
(assoc 'b '((a . 1) (b . 2) (c . 3)))
(assoc 'x '((a . 1)))
```

symbol->string

Syntax: (symbol->string 'symbol)

Wandelt ein Symbol in seinen String-Namen um.

```
(symbol->string 'foo)
(symbol->string 'my-var)
```

PostgreSQL

pg-connect

Syntax: (pg-connect "connection-string")

Öffnet eine PostgreSQL-Verbindung. Connection-String z.B.
host=... port=... user=... password=... dbname=...

```
(define conn (pg-connect "host=localhost user=golisp
                          dbname=test sslmode=disable"))
```

pg-query

Syntax: (pg-query conn "SELECT ..." param ...)

Führt eine SELECT-Abfrage aus und gibt eine Liste von
Zeilen (Assoziationslisten) zurück.

```
(pg-query conn "SELECT * FROM users WHERE id = $1" 42)
```

pg-exec

Syntax: (pg-exec conn "INSERT/UPDATE ..." param ...)

Führt eine schreibende Abfrage aus und gibt die Anzahl
betroffener Zeilen zurück.

```
(pg-exec conn "INSERT INTO users (name) VALUES ($1)"
           "Alice")
```

pg-close

Syntax: (pg-close conn)

Schließt eine PostgreSQL-Verbindung.

```
(pg-close conn)
```

Standardbibliothek

cadr

Syntax: (cadr liste)

Kurzform für (car (cdr liste)).

```
(cadr '(1 2 3))  
(cadr '(a b c))
```

caddr

Syntax: (caddr liste)

Kurzform für (car (cdr (cdr liste))).

```
(caddr '(1 2 3))  
(caddr '(a b c d))
```

caddr

Syntax: (caddr liste)

Kurzform für (car (cdr (cdr (cdr liste)))).

```
(caddr '(1 2 3 4))
```

cddr

Syntax: (cddr liste)

Kurzform für (cdr (cdr liste)).

```
(cddr '(1 2 3))  
(cddr '(a b c d))
```

cdar

Syntax: (cdar liste)

Kurzform für (cdr (car liste)).

```
(cdar '((a b c)))
```


caar

Syntax: (caar liste)

Kurzform für (car (car liste)).

```
(caar '((a b)))
```

first

Syntax: (first liste)

Alias für car.

```
(first '(a b c))
```

second

Syntax: (second liste)

Alias für cadr.

```
(second '(a b c))
```

third

Syntax: (third liste)

Alias für caddr.

```
(third '(a b c))
```

fourth

Syntax: (fourth liste)

Alias für caddr.

```
(fourth '(a b c d))
```

rest

Syntax: (rest liste)

Alias für cdr.

```
(rest '(a b c))
```

zero?

Syntax: (zero? n)

Gibt t zurück, wenn n gleich 0 ist.

```
(zero? 0)
```

```
(zero? 5)
```

positive?

Syntax: (positive? n)

Gibt t zurück, wenn n größer als 0 ist.

```
(positive? 5)
```

```
(positive? -1)
```

negative?

Syntax: (negative? n)

Gibt t zurück, wenn n kleiner als 0 ist.

```
(negative? -3)
```

```
(negative? 1)
```

pair?

Syntax: (pair? x)

Gibt t zurück, wenn x ein Cons / keine Atom ist.

```
(pair? '(1 2))
```

```
(pair? 5)
```

when

Syntax: (when test body ...)

Makro: wertet body nur aus, wenn test wahr ist.

```
(when t 'ja)
```

```
(when nil 'nein)
```

unless

Syntax: (unless test body ...)

Makro: wertet body nur aus, wenn test falsch ist.

```
(unless nil 'ja)  
(unless t 'nein)
```

append

Syntax: (append lst1 lst2)

Hängt zwei Listen aneinander.

```
(append '(1 2) '(3 4))  
(append '(a) '(b c d))
```

reverse

Syntax: (reverse liste)

Dreht eine Liste um.

```
(reverse '(1 2 3))  
(reverse '(a b c d))
```

length

Syntax: (length liste)

Liefert die Anzahl der Elemente einer Liste.

```
(length '(1 2 3))  
(length '())
```

nth

Syntax: (nth n liste)

Liefert das n-te Element einer Liste (0-basiert).

```
(nth 0 '(a b c))  
(nth 2 '(a b c))
```

last

Syntax: (last liste)

Liefert das letzte Element einer Liste.

```
(last '(1 2 3))  
(last '(a b))
```

member

Syntax: (member x liste)

Sucht x in einer Liste und gibt die Restliste ab dem Treffer zurück.

```
(member 'b '(a b c))  
(member 'x '(a b c))
```

assoc

Syntax: (assoc key liste)

Sucht key in einer Assoziationsliste und gibt das erste passende (key val)-Paar zurück.

```
(assoc 'b '((a 1) (b 2) (c 3)))  
(assoc 'x '((a 1)))
```

filter

Syntax: (filter pred liste)

Liefert alle Elemente, für die die Prädikatfunktion wahr ist.

```
(filter (lambda (x) (> x 2)) '(1 2 3 4))  
(filter atom? '((1 2) a (3 4) b))
```

reduce

Syntax: (reduce f init liste)

Faltet eine Liste mit einer Funktion und einem Startwert zusammen.

```
(reduce + 0 '(1 2 3 4))  
(reduce * 1 '(1 2 3 4))
```

for-each

Syntax: (for-each f liste)

Wendet f auf jedes Element an, gibt nil zurück.

```
(for-each println '(1 2 3))  
(for-each (lambda (x) (print (* x x))) '(2 3))
```

any

Syntax: (any pred liste)

Gibt t zurück, wenn mindestens ein Element die Prädikatfunktion erfüllt.

```
(any (lambda (x) (> x 5)) '(1 2 6))  
(any (lambda (x) (> x 5)) '(1 2 3))
```

every

Syntax: (every pred liste)

Gibt t zurück, wenn alle Elemente die Prädikatfunktion erfüllen.

```
(every number? '(1 2 3))  
(every number? '(1 a 3))
```

flatten

Syntax: (flatten liste)

Entfernt eine Ebene von Verschachtelungen; rekursiv flach.

```
(flatten '((1 2) (3 (4))))  
(flatten '(a (b c) d))
```

zip

Syntax: (zip lst1 lst2)

Verbindet zwei Listen elementweise zu Paaren.

```
(zip '(1 2) '(a b))  
(zip '(1 2 3) '(a b))
```

list-tail

Syntax: (list-tail liste n)

Liefert den n-ten Rest der Liste.

```
(list-tail '(1 2 3) 0)  
(list-tail '(1 2 3) 2)
```

iota

Syntax: (iota n)

Erzeugt die Liste (0 1 ... n-1).

```
(iota 5)  
(iota 0)
```

square

Syntax: (square x)

Quadriert x.

```
(square 5)  
(square 3)
```

expt

Syntax: (expt base exp)

Berechnet base hoch exp (exp ganze Zahl, rekursiv).

```
(expt 2 10)  
(expt 3 3)
```

gcd

Syntax: (gcd a b)

Berechnet den größten gemeinsamen Teiler.

```
(gcd 48 18)
(gcd 100 35)
```

dotimes

Syntax: (dotimes (var n) body ...)

Makro: wiederholt body für var von 0 bis n-1.

```
(dotimes (i 3) (print i))
(dotimes (i 3) (println (* i i)))
```

dolist

Syntax: (dolist (var liste) body ...)

Makro: iteriert über eine Liste.

```
(dolist (x '(1 2 3)) (print x))
(dolist (c '("a" "b")) (println c))
```

push

Syntax: (push wert var)

Makro: fügt wert am Anfang der Liste in var ein.

```
(define s '())
(push 1 s)
(push 2 s)
s
```

pop

Syntax: (pop var)

Makro: entfernt und liefert das erste Element der Liste in var.

```
(define s '(1 2 3))
(pop s)
s
```

alist-set

Syntax: (alist-set key val liste)

Setzt oder ersetzt einen Eintrag in einer Assoziationsliste.

```
(alist-set 'b 99 '((a 1) (b 2)))  
(alist-set 'd 4 '((a 1)))
```

alist-get

Syntax: (alist-get key liste)

Liefert den Wert zu key aus einer Assoziationsliste oder nil.

```
(alist-get 'b '((a 1) (b 2)))  
(alist-get 'x '((a 1)))
```

identity

Syntax: (identity x)

Gibt x unverändert zurück.

```
(identity 42)  
(identity '(1 2 3))
```

constantly

Syntax: (constantly x)

Liefert eine Funktion, die bei jedem Aufruf x zurückgibt.

```
((constantly 5) 'a 'b)  
((constantly 'ok) 1 2 3)
```

complement

Syntax: (complement f)

Liefert das logische Komplement einer Funktion.

```
((complement null?) '(1 2))  
((complement zero?) 5)
```


compose

Syntax: (compose f g)

Liefert eine Funktion, die $f(g(x))$ auf ein Argument anwendet.

```
((compose number->string (lambda (x) (* x 2))) 5)  
((compose string-upcase symbol->string) 'foo)
```